

هانیه وطنی

توضیح کارکرد الگوریتم غیر بازگشتی

1400012268035

ابتدا یک ورودی از نوع String را از طرف کاربر دریافت می‌کنیم. تابع `printText` یک آرگومان ورودی String `text` که همان input باشد نیاز دارد.

تابع `printText`:

ابتدا دو stack جدید می‌سازیم. یک stack اعداد شمارنده پشت پرانتز، و دیگری index نقطه شروع پرانتز را نگه داری می‌کند. یک متغیر از نوع int به نام index می‌سازیم و مقدار اولیه آن را 0 وارد می‌کنیم. یک حلقه `while` می‌سازیم با شرط اینکه آیا index کوچک تر از اندازه text هست یا نه. با استفاده از `switch-case` هر کارکتر text را بررسی می‌کنیم.

۱. اگر کارکتر برابر با '(' باشد یعنی loop جدید ایجاد شده که باید یک نقطه شروع به لیست start اضافه شود. همچنین در اینجا یک شرط برای بررسی اینکه آیا عددی پشت پرانتز وجود دارد یا نه گذاشته شده؛ که اگر پشت پرانتز عددی نباشد یعنی تعداد برابر است با ۱ که آن را به لیست خود اضافه می‌کنیم. سپس index را یکی بیشتر می‌کنیم.

۲. اگر کارکتر برابر با ')' باشد یعنی loop ما به پایان رسیده. باید چک کنیم که آیا تعداد بارهایی که این loop چاپ شده برابر است با عددی که پشت پرانتز درج شده. برای این کار هر باری که کارکتر '(' ظاهر شود آخرین عضو لیست count برداشته شده و یکی کمتر از آن را به stack اضافه می‌کنیم. اگر آخرین عضو لیست بزرگتر از ۱ باشد از عدد آخر count یکی کم می‌کنیم و به index نقطه شروع (آخرین عضو لیست start) می‌دهیم. اگر این چنین نبود آخرین عضو هر دو لیست را از لیست خارج می‌کنیم و index را یکی اضافه می‌کنیم.

۳. اگر کارکتر هیچ کدام از گزینه های بالا نباشد باید چک کنیم که آیا کارکتر عدد است یا خیر. اگر عدد باشد باید چک کنیم که دو رقمی است یا نه. سپس عدد را به لیست count اضافه کرده و بنا به تعداد ارقام عدد پشت پرانتز به index یکی یا دو تا اضافه می‌کنیم.

۴. اگر کارکتر عدد نباشد و حروف انگلیسی باشد باید کارکتر را چاپ کرده و به index یکی اضافه می‌کنیم.

یک تابع boolean به عنوان `intCheck` هم تعریف می‌کنیم که چک کند آیا کارکتر جزوی از اعداد 0 تا 9 هست یا نه.

محاسبه پیچیدگی: از آن جایی که پیچیدگی Stack برابر است با $O(1)$ بنابراین پیچیدگی زمانی برنامه برابر میشود با اندازه استرینگ وارد شده؛ که می‌شود $O(n)$.